

C++ Concurrent Queues

Committee: ISO/IEC JTC1 SC22 WG21 SG1 Concurrency

Document Number: P0260r0

Date: 2016-02-14

Authors: Lawrence Crowl, Chris Mysisen

Reply To: Lawrence Crowl, Lawrence@Crowl.org

[Introduction](#)

[Conceptual Interface](#)

[Basic Operations](#)

[Non-Waiting Operations](#)

[Non-Blocking Operations](#)

[Push Front Operations](#)

[Closed Queues](#)

[Empty and Full Queues](#)

[Queue Names](#)

[Element Type Requirements](#)

[Exception Handling](#)

[Queue Ordering](#)

[Lock-Free Implementations](#)

[Concrete Queues](#)

[Locking Buffer Queue](#)

[Lock-Free Buffer Queue](#)

[Additional Conceptual Tools](#)

[Fronts and Backs](#)

[Streaming Iterators](#)

[Storage Iterators](#)

[Binary Interfaces](#)

[Managed Indirection](#)

[Implementation](#)

[Proposed Wording](#)

[?? Concurrent queues \[conqueues\]](#)

[??1 General \[conqueues.general\]](#)

[??2 Header <conqueue> synopsis \[conqueues.syn\]](#)

[??3 Operation status \[conqueues.status\]](#)

[??4 Concepts \[conqueues.concepts\]](#)

[??4.1 Element requirements \[conqueues.concept.elemreq\]](#)

[??4.2 Element type naming \[conqueues.concept.elemtype\]](#)

[??4.3 Lock-free attribute operations \[conqueues.concept.lockfree\]](#)

[??4.4 State operations \[conqueues.concept.state\]](#)

[??4.5 Waiting operations \[conqueues.concept.wait\]](#)

[??4.6 Non-waiting operations \[conqueues.concept.nonwait\]](#)

[??4.7 Non-blocking operations \[conqueues.concept.nonblock\]](#)

[??4.8 Type concepts \[conqueues.concept.type\]](#)

[??5 Concrete queues \[conqueues.concrete\]](#)

[??5.1 Class template buffer_queue \[conqueues.concrete.buffer\]](#)

[??5.2 Class template lock_free_buffer_queue \[conqueues.concrete.lockfree\]](#)

[??6 Tools \[conqueues.tools\]](#)

[??6.1 Ends and Iterators \[conqueues.tools.ends\]](#)

[??6.1.1 Class template generic_queue_back \[conqueues.tools.back\]](#)

[??6.1.2 Class template generic_queue_front \[conqueues.tools.front\]](#)

[??6.2 Binary interfaces \[conqueues.tools.binary\]](#)

[??6.2.1 Class template queue_base \[conqueues.tools.base\]](#)

[??6.2.2 Class template queue_wrapper \[conqueues.tools.wrap\]](#)

[??6.2.3 Binary ends \[conqueues.tools.binends\]](#)

[??6.3 Managed Ends \[conqueues.tools.managed\]](#)

[??6.3.1 Class template shared_queue_back \[conqueues.tools.sharedback\]](#)

[??6.3.2 Class template shared_queue_front \[conqueues.tools.front\]](#)

[??6.3.3 Function template share_queue_ends \[conqueues.tools.shareendsfront\]](#)

Introduction

Queues provide a mechanism for communicating data between components of a system.

The existing deque in the standard library is an inherently sequential data structure. Its reference-returning element access operations cannot synchronize access to those elements with other queue operations. So, concurrent pushes and pops on queues require a different interface to the queue structure.

Moreover, concurrency adds a new dimension for performance and semantics. Different queue implementation must trade off uncontended operation cost, contended operation cost, and element order guarantees. Some of these trade-offs will necessarily result in semantics weaker than a serial queue.

Conceptual Interface

We provide basic queue operations, and then extend those operations to cover other important issues.

Basic Operations

The essential solution to the problem of concurrent queuing is to shift to value-based operations, rather than reference-based operations.

The basic operations are:

```
void queue::push(const Element&);  
void queue::push(Element&&);
```

Push the *Element* onto the queue.

```
Element queue::value_pop();
```

Pop an *Element* from the queue. The element will be moved out of the queue in preference to being copied.

These operations will wait when the queue is full or empty. (Not all queue implementations can actually be full.) These operations may block for mutual exclusion as well.

Non-Waiting Operations

Waiting on a full or empty queue can take a while, which has an opportunity cost. Avoiding that wait enables algorithms to avoid queuing speculative work when a queue is full, to do other work rather than wait for a push on a full queue, and to do other work rather than wait for a pop on an empty queue.

```
queue_op_status queue::try_push(const Element&);  
queue_op_status queue::try_push(Element&&);
```

If the queue is full, return `queue_op_status::full`. Otherwise, push the *Element* onto the queue. Return `queue_op_status::success`.

```
queue_op_status queue::try_pop(Element&);
```

If the queue is empty, return `queue_op_status::empty`. Otherwise, pop the *Element* from the queue. The element will be moved out of the queue in preference to being copied. Return `queue_op_status::success`.

These operations will not wait when the queue is full or empty. They may block for mutual exclusion.

Non-Blocking Operations

For cases when blocking for mutual exclusion is undesirable, we have non-blocking operations. The interface is the same as the try operations but is allowed to also return `queue_op_status::busy` in case the operation is unable to complete without blocking.

```
queue_op_status queue::nonblocking_push(const Element&);  
queue_op_status queue::nonblocking_push(Element&&);
```

If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is full, return `queue_op_status::full`. Otherwise, push the *Element* onto the queue. Return `queue_op_status::success`.

```
queue_op_status queue::nonblocking_pop(Element&);
```

If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is empty, return `queue_op_status::empty`. Otherwise, pop the *Element* from the queue. The element will be moved out of the queue in preference to being copied. Return `queue_op_status::success`.

These operations will neither wait nor block. However, they may do nothing.

The non-blocking operations highlight a terminology problem. In terms of synchronization effects, `nonwaiting_push` on queues is equivalent to `try_lock` on mutexes. And so one could conclude that the existing `try_push` should be renamed `nonwaiting_push` and `nonblocking_push` should be renamed `try_push`. However, at least Thread Building Blocks uses the existing terminology. Perhaps better is to not use `try_push` and instead use `nonwaiting_push` and `nonblocking_push`.

Push Front Operations

Occasionally, one may wish to return a popped item to the queue. We can provide for this with `push_front` operations.

```
void queue::push_front(const Element&);  
void queue::push_front(Element&&);
```

Push the *Element* onto the back of the queue, i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.

```
queue_op_status queue::try_push_front(const Element&);  
queue_op_status queue::try_push_front(Element&&);
```

If the queue was full, return `queue_op_status::full`. Otherwise, push the *Element* onto the front of the queue, i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.

```
queue_op_status queue::nonblocking_push_front(const Element&);  
queue_op_status queue::nonblocking_push_front(Element&&);
```

If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is full, return `queue_op_status::full`. Otherwise, push the *Element* onto the front queue. i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.

This feature was requested at the Spring 2012 meeting. However, we do not think the feature works.

- The name `push_front` is inconsistent with existing "push back" nomenclature.
- The effects of `push_front` are only distinguishable from a regular push when there is a strong ordering of elements. Highly concurrent queues will likely have no strong ordering.
- The `push_front` call may fail due to full queues, closed queues, etc. In which case the operation will suffer contention, and may succeed only after interposing push and pop operations. The consequence is that the original push order is not preserved in the final pop order. So, `push_front` cannot be directly used as an 'undo'.
- The operation implies an ability to reverse internal changes at the front of the queue. This ability implies a loss efficiency in some implementations.

In short, we do not think that in a concurrent environment `push_front` provides sufficient semantic value to justify its cost. Consequently, the proposed wording does not provide this feature.

Closed Queues

Threads using a queue for communication need some mechanism to signal when the queue is no longer needed. The usual approach is add an additional out-of-band signal. However, this approach suffers from the flaw that threads waiting on either full or empty queues need to be woken up when the queue is no longer needed. To do that, you need access to the condition variables used for full/empty blocking, which considerably increases the complexity and fragility of the interface. It also leads to performance implications with additional mutexes or atomics. Rather than require an out-of-band signal, we chose to directly support such a signal in the queue itself, which considerably simplifies coding.

To achieve this signal, a thread may *close* a queue. Once closed, no new elements may be pushed onto the queue. Push operations on a closed queue will either return `queue_op_status::closed` (when they have a status return) or throw `queue_op_status::closed` (when they do not). Elements already on the queue may be popped off. When a queue is empty and closed, pop operations will either return `queue_op_status::closed` (when they have a status return) or throw `queue_op_status::closed` (when they do not).

The additional operations are as follows. They are essentially equivalent to the basic operations except that they return a status, avoiding an exception when queues are closed.

```
void queue::close();
```

Close the queue.

```
bool queue::is_closed();
```

Return true iff the queue is closed.

```
queue_op_status queue::wait_push(const Element&);
```

```
queue_op_status queue::wait_push(Element&&);
```

If the queue was closed, return `queue_op_status::closed`. Otherwise, push the *Element* onto the queue. Return `queue_op_status::success`.

```
queue_op_status queue::wait_pop(Element&);
```

If the queue is empty and closed, return `queue_op_status::closed`. Otherwise, if the queue is empty, return `queue_op_status::empty`. Otherwise, pop the *Element* from the queue. The element will be moved out of the queue in preference to being copied. Return `queue_op_status::success`.

The push and pop operations will wait when the queue is full or empty. All these operations may block for mutual exclusion as well.

There are use cases for opening a queue that is closed. While we are not aware of an implementation in which the ability to reopen a queue would be a hardship, we also imagine that such an implementation could exist. Open should generally only be called if the queue is closed and empty, providing a clean synchronization point, though it is possible to call open on a non-empty queue. An open operation following a close operation is guaranteed to be visible after the close operation and the queue is guaranteed to be open upon completion of the open call. (But of course, another close call could occur immediately thereafter.)

```
void queue::open();
```

Open the queue.

Note that when `is_closed()` returns false, there is no assurance that any subsequent operation finds the queue closed because some other thread may close it concurrently.

If an open operation is not available, there is an assurance that once closed, a queue stays closed. So, unless the programmer takes care to ensure that all other threads will not close the queue, only a return value of true has any meaning.

Given these concerns with reopening queues, we do not propose wording to reopen a queue.

Empty and Full Queues

It is sometimes desirable to know if a queue is empty.

```
bool queue::is_empty();
```

Return true iff the queue is empty.

This operation is useful only during intervals when the queue is known to not be subject to pushes and pops from other threads. Its primary use case is assertions on the state of the queue at the end of its lifetime, or when the system is in quiescent state (where there no outstanding pushes).

We can imagine occasional use for knowing when a queue is full, for instance in system performance polling. The motivation is significantly weaker though.

```
bool queue::is_full();
```

Return true iff the queue is full.

Not all queues will have a full state, and these would always return false.

Queue Names

It is sometimes desirable for queues to be able to identify themselves. This feature is particularly helpful for run-time diagnostics, particularly when 'ends' become dynamically passed around between threads. See [Managed Indirection](#) below.

```
const char* queue::name();
```

Return the name string provided as a parameter to queue construction.

There is some debate on this facility, but we see no way to effectively replicate the facility. However, in recognition of that debate, the wording below does not provide the name facility.

Element Type Requirements

The above operations require element types with a default constructor, copy/move constructors, copy/move assignment operators, and destructor. These operations may be trivial. The default constructor and destructor shall not throw. The copy/move constructors and copy/move assignment operators may throw, but must leave the objects in a valid state for subsequent operations.

Exception Handling

Concurrent queues cannot completely hide the effect of exceptions, in part because changes cannot be transparently undone when other threads are observing the queue.

Queues may rethrow exceptions from storage allocation, mutexes, or condition variables.

If the [element type operations required](#) do not throw exceptions, then only the exceptions above are rethrown.

When an element copy/move may throw, some queue operations have additional behavior.

- Construction shall rethrow, destroying any elements allocated.
- A push operation shall rethrow and the state of the queue is unaffected.
- A pop operation shall rethrow and the element is popped from the queue. The value popped is effectively lost. (Doing otherwise would likely clog the queue with a bad element.)

Queue Ordering

The conceptual queue interface makes minimal guarantees.

- The queue is not empty if there is an element that has been pushed but not popped.
- A push operation *synchronizes with* the pop operation that obtains that element.
- A close operation *synchronizes with* an operation that observes that the queue is closed.
- There is a sequentially consistent order of operations.

In particular, the conceptual interface does not guarantee that the sequentially consistent order of element pushes matches the sequentially consistent order of pops. Concrete queues could specify more specific ordering guarantees.

Lock-Free Implementations

Lock-free queues will have some trouble waiting for the queue to be non-empty or non-full. Therefore, we propose two closely-related concepts. A full concurrent queue concept as described above, and a non-waiting concurrent queue concept that has all the operations except push, wait_push, value_pop and wait_pop. That is, it has only non-waiting operations (presumably emulated with busy wait) and non-blocking operations, but no waiting operations. We propose naming these `WaitingConcurrentQueue` and `NonWaitingConcurrentQueue`, respectively.

Note: Adopting this conceptual split requires splitting some of the facilities defined later.

It may be helpful to know if a concurrent queue has a lock free implementation.

```
bool queue::is_lock_free();
```

Return true iff the has a lock-free implementation.

Concrete Queues

In addition to the concept, the standard needs at least one concrete queue. We describe two concrete queues.

Locking Buffer Queue

We provide a concrete concurrent queue in the form of a fixed-size `buffer_queue`. It meets the `WaitingConcurrentQueue` concept. It provides for construction of an empty queue, and construction of a queue from a pair of iterators. Constructors take a parameter specifying the maximum number of elements in the buffer. Constructors may also take a parameter specifying the name of the queue. If the name is not present, it defaults to the empty string.

The `buffer_queue` deletes the default constructor, the copy constructor, and the copy assignment operator. We feel that their benefit might not justify their potential confusion.

Lock-Free Buffer Queue

We provide a concrete concurrent queue in the form of a fixed-size `lock_free_buffer_queue`. It meets the `NonWaitingConcurrentQueue` concept. The queue is still under development, so details may change.

Additional Conceptual Tools

There are a number of tools that support use of the conceptual interface. These tools are not part of the queue interface, but provide restricted views or adapters on top of the queue useful in implementing concurrent algorithms.

Fronts and Backs

Restricting an interface to one side of a queue is a valuable code structuring tool. This restriction is accomplished with the classes `generic_queue_front` and `generic_queue_back` parameterized on the concrete queue implementation. These act as pointers with access to only the front or the back of a queue. The front of the queue is where elements are popped. The back of the queue is where elements are pushed.

```
void send( int number, generic_queue_back<buffer_queue<int>> arv );
```

These fronts and backs are also able to provide `begin` and `end` operations that unambiguously stream data into or out of a queue.

Streaming Iterators

In order to enable the use of existing algorithms streaming through concurrent queues, they need to support iterators. Output iterators will push to a queue and input iterators will pop from a queue. Stronger forms of iterators are in general not possible with concurrent queues.

Iterators implicitly require waiting for the advance, so iterators are only supportable with the `WaitingConcurrentQueue` concept.

```
void iterate(
    generic_queue_back<buffer_queue<int>>::iterator bitr,
    generic_queue_back<buffer_queue<int>>::iterator bend,
    generic_queue_front<buffer_queue<int>>::iterator fitr,
    generic_queue_front<buffer_queue<int>>::iterator fend,
    int (*compute)( int ) )
{
    while ( fitr != fend && bitr != bend )
        *bitr++ = compute(*fitr++);
}
```

Note that contrary to existing iterator algorithms, we check both iterators for reaching their end, as either may be closed at any time.

Note that with suitable renaming, the existing standard front insert and back insert iterators could work as is. However, there is nothing like a pop iterator adapter.

Storage Iterators

In addition to iterators that stream data into and out of a queue, we could provide an iterator over the storage contents of a queue. Such an iterator, even when implementable, would mostly likely be valid only when the queue is otherwise quiescent. We believe such an iterator would be most useful for debugging, which may well require knowledge of the concrete class. Therefore, we do not propose wording for this feature.

Binary Interfaces

The standard library is template based, but it is often desirable to have a binary interface that shields client from the concrete implementations. For example, `std::function` is a binary interface to callable object (of a given signature). We achieve this

capability in queues with type erasure.

We provide a `queue_base` class template parameterized by the value type. Its operations are virtual. This class provides the essential independence from the queue representation.

We also provide `queue_front` and `queue_back` class templates parameterized by the value types. These are essentially `generic_queue_front<queue_base<Value>>` and `generic_queue_back<queue_base<Value>>`, respectively.

To obtain a pointer to `queue_base` from an non-virtual concurrent queue, construct an instance the `queue_wrapper` class template, which is parameterized on the queue and derived from `queue_base`. Upcasting a pointer to the `queue_wrapper` instance to a `queue_base` instance thus erases the concrete queue type.

```
extern void seq_fill( int count, queue_back<int> b );

buffer_queue<int> body( 10 /*elements*/, /*named*/ "body" );
queue_wrapper<buffer_queue<int>> wrap( body );
seq_fill( 10, wrap.back() );
```

Managed Indirection

Long running servers may have the need to reconfigure the relationship between queues and threads. The ability to pass 'ends' of queues between threads with automatic memory management eases programming.

To this end, we provide `shared_queue_front` and `shared_queue_back` template classes. These act as reference-counted versions of the `queue_front` and `queue_back` template classes. These shared versions act on a `queue_counted` class template, which is a counted version of `queue_base`.

The concrete counted queues are the `queue_owner` class template, which takes ownership of a raw pointer to a queue, and the `queue_object` class template, which constructs and instance of the implementation queue within itself. Both class templates are derived from `queue_counted`.

```
queue_owner<buffer_queue<int>> own( new buffer_queue<int>(10, "own") );
seq_fill( 10, own.back() );
queue_object<buffer_queue<int>> obj( 10, "own" );
seq_fill( 10, obj.back() );
```

The `share_queue_ends(Args ... args)` template function will provide a pair of `shared_queue_front` and `shared_queue_back` to a dynamically allocated `queue_object` instance containing an instance of the specified implementation queue. When the last of these fronts and backs are deleted, the queue itself will be deleted. Also, when the last of the fronts or the last of the backs is deleted, the queue will be closed.

```
auto x = share_queue_ends<buffer_queue<int>>( 10, "shared" );
shared_queue_front<int> f(x.first);
shared_queue_back<int> b(x.second);
f.push(3);
assert(3 == b.value_pop());
```

Implementation

A free, open-source implementation of these interfaces is available at the Google Concurrency Library project at <https://github.com/aldasairmackintosh/google-concurrency-library>. The concrete `buffer_queue` is in [.../blob/master/include/buffer_queue.h](https://github.com/aldasairmackintosh/google-concurrency-library/blob/master/include/buffer_queue.h). The concrete `lock_free_buffer_queue` is in [.../blob/master/include/lock_free_buffer_queue.h](https://github.com/aldasairmackintosh/google-concurrency-library/blob/master/include/lock_free_buffer_queue.h). The corresponding implementation of the conceptual tools is in [.../blob/master/include/queue_base.h](https://github.com/aldasairmackintosh/google-concurrency-library/blob/master/include/queue_base.h).

Proposed Wording

The concurrent queue container definition is as follows. The section, paragraph, and table references are based on those of [N4567 Working Draft, Standard for Programming Language C++](#), Richard Smith, November 2015.

?.? Concurrent queues [conqueues]

Add a new section.

?.?.1 General [conqueues.general]

Add a new section.

This section provides mechanisms for concurrent access to a queue. These mechanisms ease the production of race-free programs (1.10 [intro.multithread]).

?.?.2 Header <conqueue> synopsis [conqueues.syn]

To Be Determined.

?.?.3 Operation status [conqueues.status]

Add a new section.

Many concurrent queue operations return a status in the form of the following enumeration.

enum class queue_op_status

Enumerators: success = 0, empty, full, closed, busy

?.?.4 Concepts [conqueues.concepts]

Add a new section.

This section provides the conceptual operations for concurrent queues of type *queue* of *Element* types.

?.?.4.1 Element requirements [conqueues.concept.elemreq]

Add a new section:

The types of the elements of a concurrent queue must provide a default constructor, either or both of a copy constructor or a move constructor, either or both of a copy assignment operator or a move assignment operator, and a destructor.

The default constructor and destructor shall not throw. Any copy/move constructor or copy/move assignment operator that throws shall leave the objects in a valid state for subsequent operations.

?.?.4.2 Element type naming [conqueues.concept.elemtype]

Add a new section:

The queue class shall provide a typedef to its element value type.

typedef *implementation-defined* value_type;

?.?.4.3 Lock-free attribute operations [conqueues.concept.lockfree]

Add a new section:

A queue type provides lock-free operations (1.10 [intro.multithread], or it does not.

bool *queue*::is_lock_free();

Returns: If the operations of the queue type are lock-free, true. Otherwise, false.

Remark: The function returns the same result for all instances of the type.

??4.4 State operations [conqueues.concept.state]

Add a new section:

Upon construction, every queue shall be in an open state. It may move to a closed state, but shall not move back to an open state.

void queue::close();

Effects: Closes the queue. No pushes subsequent to the close shall succeed.

Throws: Nothing.

Synchronization: A close operation synchronizes-with any operation that observes the closed state.

bool queue::is_closed();

Returns: true if the queue is closed, otherwise, false

Throws: Nothing.

Synchronization: See the close operation.

bool queue::is_empty();

Returns: true if every element pushed has also been popped. Otherwise, false

Throws: Nothing.

Synchronization: None.

Remark: This operation is typically useful only when threads are known to be in a state where they will not push.

bool queue::is_full();

Returns: true if a push operation would fail due to unavailable space. Otherwise, false

Throws: Nothing.

Synchronization: None.

Remark: This operation is typically useful only when threads are known to be in a state where they will not pop.

??4.5 Waiting operations [conqueues.concept.wait]

Add a new section:

void queue::push(const Element&);

void queue::push(Element&&);

Effects: If space is available on the queue, pushes the *element* onto the queue. If space is not available, waits until space is available.

Throws: Any exception from operations on storage allocation, mutexes, or condition variables. If a copy/move operation throws, the state of the queue is unaffected and the push shall rethrow the exception.

Synchronization: A push operation synchronizes-with the operation that pops the corresponding element. See the close operation.

Element queue::pop();

Effects: If an element is available on the queue, moves the *element* from the queue to the result. Otherwise, if the queue is closed, throws an exception. Otherwise, waits until an element is available.

Returns: The element moved from the queue.

Throws: Any exception from operations on storage allocation, mutexes, or condition variables. If an element copy/move operation throws, the state of the element is popped and the pop shall rethrow the exception.

Synchronization: There is a sequentially consistent order of elements popped.

```
queue_op_status queue::wait_push(const Element&);
```

```
queue_op_status queue::wait_push(Element&&);
```

Effects: If space is available on the queue, pushes the *element* onto the queue. If space is not available, waits until space is available.

Returns: If the queue is closed, `queue_op_status::closed`. Otherwise, the push was successful, `queue_op_status::success`.

Throws: Any exception from operations on storage allocation, mutexes, or condition variables. If a copy/move operation throws, the state of the queue is unaffected and the push shall rethrow the exception.

Synchronization: There is a sequentially consistent order of elements pushed. A push operation synchronizes with the operation that pops the corresponding element. See the `close` operation.

```
queue_op_status queue::wait_pop(Element&);
```

Effects: If an element is available on the queue, moves the *element* from the queue to the parameter.

Returns: If the queue is closed, `queue_op_status::closed`. Otherwise, if the pop was successful, `queue_op_status::success`. Otherwise, no element is available, waits for one to become available.

Throws: Any exception from operations on storage allocation, mutexes, or condition variables. If an element copy/move operation throws, the state of the element is popped and the pop shall rethrow the exception.

Synchronization: There is a sequentially consistent order of elements returned by pop operations. See the push operations. See the `close` operation.

?.?.4.6 Non-waiting operations [`conqueues.concept.nonwait`]

Add a new section:

```
queue_op_status queue::try_push(const Element&);
```

```
queue_op_status queue::try_push(Element&&);
```

Effects: If space is available on the queue, copies or moves the *element* onto the queue. If space is not available, return `queue_op_status::full`.

Returns: If the queue is closed, `queue_op_status::closed`. Otherwise, if the push was successful, `queue_op_status::success`. Otherwise, space is unavailable, `queue_op_status::full`.

Throws: Any exception from operations on storage allocation, mutexes, or condition variables. If a copy/move operation throws, the state of the queue is unaffected and the push shall rethrow the exception.

Synchronization: There is a sequentially consistent order of elements pushed. A push operation synchronizes with the operation that pops the corresponding element. See the `close` operation.

```
queue_op_status queue::try_pop(Element&);
```

Effects: If an element is available on the queue, moves the *element* from the queue into the parameter.

Returns: If the queue is closed, `queue_op_status::closed`. Otherwise, if the pop was successful, `queue_op_status::success`. Otherwise, no element is available, `queue_op_status::empty`.

Throws: Any exception from operations on storage allocation, mutexes, or condition variables. If an element copy/move operation throws, the state of the element is popped and the pop shall rethrow the exception.

Synchronization: There is a sequentially consistent order of elements returned by pop operations. See the push operations. See the `close` operation.

?.?.4.7 Non-blocking operations [conqueues.concept.nonblock]

Add a new section:

```
queue_op_status queue::nonblocking_push(const Element&);  
queue_op_status queue::nonblocking_push(Element&&);
```

Effects: If space is immediately available on the queue, pushes the *element* onto the queue. If space is not immediately available, return `queue_op_status::busy`. If space is not available, return `queue_op_status::full`.

Returns: If the queue is closed, `queue_op_status::closed`. Otherwise, if the push was successful, `queue_op_status::success`. Otherwise, space is immediately unavailable, `queue_op_status::busy`. Otherwise, space is unavailable, `queue_op_status::full`.

Throws: Any exception from operations on storage allocation, mutexes, or condition variables. If a copy/move operation throws, the state of the queue is unaffected and the push shall rethrow the exception.

Synchronization: There is a sequentially consistent order of elements pushed. A push operation synchronizes with the operation that pops the corresponding element. See the `close` operation.

```
queue_op_status queue::nonblocking_pop(Element&);
```

Effects: If an element is immediately available on the queue, moves the *element* from the queue into the parameter.

Returns: If the queue is closed, `queue_op_status::closed`. Otherwise, if the pop was successful, `queue_op_status::success`. Otherwise, no element is immediately available, `queue_op_status::busy`. Otherwise, no element is available, `queue_op_status::empty`.

Throws: Any exception from operations on storage allocation, mutexes, or condition variables. If an element copy/move operation throws, the state of the element is popped and the pop shall rethrow the exception.

Synchronization: There is a sequentially consistent order of elements returned by pop operations. See the push operations. See the `close` operation.

?.?.4.8 Type concepts [conqueues.concept.type]

Add a new section:

The `WaitingConcurrentQueue` concept provides all of the operations specified above.

The `NonWaitingConcurrentQueue` concept provides all of the operations specified above, except the waiting operations ([conqueues.concept.wait]). A `NonWaitingConcurrentQueue` is lock-free (1.10 [intro.multithread]) when its member function `is_lock_free` reports true.

The `WaitingConcurrentQueueBack` concept provides all of the operations specified above except the pop operations.

The `WaitingConcurrentQueueFront` concept provides all of the operations specified above except the push operations.

The `NonWaitingConcurrentQueueBack` concept provides all of the operations specified above except the pop operations and the waiting push operations. A `NonWaitingConcurrentQueueBack` is lock-free (1.10 [intro.multithread]) when its member function `is_lock_free` reports true.

The `NonWaitingConcurrentQueueFront` concept provides all of the operations specified above except the push operations and the waiting pop operations. A `NonWaitingConcurrentQueueFront` is lock-free (1.10 [intro.multithread]) when its member function `is_lock_free` reports true.

?.?.5 Concrete queues [conqueues.concrete]

Add a new section:

There are two concrete concurrent queue types.

??5.1 Class template `buffer_queue`[conqueues.concrete.buffer]

Add a new section:

```
template <typename Value>
class buffer_queue
{
    buffer_queue() = delete;
    buffer_queue(const buffer_queue&) = delete;
    buffer_queue& operator =(const buffer_queue&) = delete;
public:
    typedef Value value_type;
    explicit buffer_queue(size_t max_elems);
    template <typename Iter>
    buffer_queue(size_t max_elems, Iter first, Iter last);
    ~buffer_queue();

    void close();
    bool is_closed();
    bool is_empty();
    bool is_full();
    bool is_lock_free();

    Value value_pop();
    queue_op status wait_pop(Value&);
    queue_op status try_pop(Value&);
    queue_op status nonblocking_pop(Value&);

    void push(const Value& x);
    queue_op status wait_push(const Value& x);
    queue_op status try_push(const Value& x);
    queue_op status nonblocking_push(const Value& x);
    void push(Value&& x);
    queue_op status wait_push(Value&& x);
    queue_op status try_push(Value&& x);
    queue_op status nonblocking_push(Value&& x);
};
```

The class template `buffer_queue` implements the `WaitingConcurrentQueue` concept.

`buffer_queue(size_t max_elems);`

Effects: Constructs the queue with a buffer capacity of `max_elems`.

Throws: Any exception thrown by constructing the internal arrays or mutexes.

template <typename Iter> `buffer_queue(size_t max_elems, Iter first, Iter last);`

Effects: Constructs the queue with a buffer capacity of `max_elems`. Initializes the internal array by iterating from `first` to `last`.

Throws: Any exception thrown by constructing the internal arrays or mutexes.

~`buffer_queue();`

Effects: Destroys the queue. Any elements remaining within the queue are discarded.

Throws: Nothing.

??5.2 Class template `lock_free_buffer_queue` [conqueues.concrete.lockfree]

Add a new section:

```
template <typename Value>
class lock_free_buffer_queue
{
    lock_free_buffer_queue() = delete;
    lock_free_buffer_queue(const lock_free_buffer_queue&) = delete;
```

```

    lock_free_buffer_queue& operator =(const lock_free_buffer_queue&) = delete;
public:
    typedef Value value_type;
    explicit lock_free_buffer_queue(size_t max_elems);
    template <typename Iter>
    lock_free_buffer_queue(size_t max_elems, Iter first, Iter last);
    ~lock_free_buffer_queue();

    void close();
    bool is_closed();
    bool is_empty();
    bool is_full();
    bool is_lock_free();

    queue_op_status try_pop(Value&);
    queue_op_status nonblocking_pop(Value&);

    queue_op_status try_push(const Value& x);
    queue_op_status nonblocking_push(const Value& x);
    queue_op_status try_push(Value&& x);
    queue_op_status nonblocking_push(Value&& x);
};

```

The class template `lock_free_buffer_queue` implements the `NonWaitingConcurrentQueue` concept.

```
lock_free_buffer_queue(size_t max_elems);
```

Effects: Constructs the queue with a buffer capacity of `max_elems`.

Throws: Any exception thrown by constructing the internal arrays or mutexes.

```
template <typename Iter> lock_free_buffer_queue(size_t max_elems, Iter first, Iter last);
```

Effects: Constructs the queue with a buffer capacity of `max_elems`. Initializes the internal array by iterating from `first` to `last`.

Throws: Any exception thrown by constructing the internal arrays or mutexes.

```
~lock_free_buffer_queue();
```

Effects: Destroys the queue. Any elements remaining within the queue are discarded.

Throws: Nothing.

???.6 Tools [conqueues.tools]

Add a new section:

Additional tools help to use and manage concurrent queues.

???.6.1 Ends and Iterators [conqueues.tools.ends]

Add a new section:

Access to only a single end of a queue is a valuable code structuring tool. A single end can also provide unambiguous begin and end operations that return iterators.

???.6.1.1 Class template `generic_queue_back` [conqueues.tools.back]

Add a new section:

```

template <typename Queue>
class generic_queue_back
{
public:
    typedef typename Queue::value_type value_type;

```

```

typedef value_type& reference;
typedef const value_type& const_reference;

typedef implementation-defined iterator;
typedef const iterator const_iterator;

generic_queue_back(Queue& queue);
generic_queue_back(Queue* queue);
generic_queue_back(const generic_queue_back& other) = default;
generic_queue_back& operator=(const generic_queue_back& other) = default;

void close();
bool is_closed();
bool is_empty();
bool is_full();
bool is_lock_free();
bool has_queue();

iterator begin();
iterator end();
const_iterator cbegin();
const_iterator cend();

void push(const value_type& x);
queue_op_status wait_push(const value_type& x);
queue_op_status try_push(const value_type& x);
queue_op_status nonblocking_push(const value_type& x);

void push(value_type&& x);
queue_op_status wait_push(value_type&& x);
queue_op_status try_push(value_type&& x);
queue_op_status nonblocking_push(value_type&& x);
};

```

The class template `generic_queue_back` implements `WaitingConcurrentQueueBack`

```

generic_queue_back(Queue& queue);
generic_queue_back(Queue* queue);

```

Effects: Constructs the queue back with a pointer to the queue object given.

```
~generic_queue_back();
```

Effects: Destroys the queue back.

```
bool has_queue();
```

Returns: true if the contained pointer is not null. false otherwise.

??6.1.2 Class template `generic_queue_front` [`conqueues.tools.front`]

Add a new section:

```

template <typename Queue>
class generic_queue_front
{
public:
    typedef typename Queue::value_type value_type;
    typedef value_type& reference;
    typedef const value_type& const_reference;

    typedef implementation-defined iterator;
    typedef const iterator const_iterator;

    generic_queue_front(Queue& queue);
    generic_queue_front(Queue* queue);
    generic_queue_front(const generic_queue_front& other) = default;
    generic_queue_front& operator=(const generic_queue_front& other) = default;

    void close();
    bool is_closed();
    bool is_empty();
    bool is_full();
    bool is_lock_free();

```

```

bool has_queue();

iterator begin();
iterator end();
const iterator cbegin();
const iterator cend();

value_type value_pop();
queue_op_status wait_pop(value_type& x);
queue_op_status try_pop(value_type& x);
queue_op_status nonblocking_pop(value_type& x);
};

```

The class template `generic_queue_front` implements `WaitingConcurrentQueueFront`

```

generic_queue_front(Queue& queue);
generic_queue_front(Queue* queue);

```

Effects: Constructs the queue front with a pointer to the queue object given.

```

~generic_queue_front();

```

Effects: Destroys the queue front.

```

bool has_queue();

```

Returns: true if the contained pointer is not null, false otherwise.

??6.2 Binary interfaces [conqueues.tools.binary]

Add a new section:

Occasionally it is best to have a binary interface to any concurrent queue of a given element type. This binary interface is provided by a base class with virtual members and a generic wrapper class that references any queue through that base class.

??6.2.1 Class template `queue_base` [conqueues.tools.base]

Add a new section:

```

template <typename Value>
class queue_base
{
public:
    typedef Value value_type;
    typedef value_type& reference;
    typedef const value_type& const_reference;

    virtual ~queue_base();

    virtual void close() = 0;
    virtual bool is_closed() = 0;
    virtual bool is_empty() = 0;
    virtual bool is_full() = 0;
    virtual bool is_lock_free() = 0;

    virtual void push(const Value& x) = 0;
    virtual queue_op_status wait_push(const Value& x) = 0;
    virtual queue_op_status try_push(const Value& x) = 0;
    virtual queue_op_status nonblocking_push(const Value& x) = 0;

    virtual void push(Value&& x) = 0;
    virtual queue_op_status wait_push(Value&& x) = 0;
    virtual queue_op_status try_push(Value&& x) = 0;
    virtual queue_op_status nonblocking_push(Value&& x) = 0;

    virtual Value value_pop() = 0;
    virtual queue_op_status wait_pop(Value&) = 0;
    virtual queue_op_status try_pop(Value&) = 0;
    virtual queue_op_status nonblocking_pop(Value&) = 0;

```

```
};
```

The class template `queue_base`, and all classes derived from it, shall implement the `WaitingConcurrentQueue` concept.

```
~queue_base();
```

Effects: Destroys the queue.

Throws: Nothing.

??6.2.2 Class template `queue_wrapper` [`conqueues.tools.wrap`]

Add a new section:

```
template <typename Queue>
class queue_wrapper
:
    public virtual queue_base <typename Queue::value_type>
{
public:
    typedef typename Queue::value_type value_type;
    typedef value_type& reference;
    typedef const value_type& const_reference;

    queue_wrapper(Queue* arg);
    queue_wrapper(Queue& arg);
    virtual ~queue_wrapper();

    virtual void close();
    virtual bool is_closed();
    virtual bool is_empty();
    virtual bool is_full() = 0;
    virtual bool is_lock_free() = 0;

    virtual void push(const value_type& x);
    virtual queue_op_status wait_push(const value_type& x);
    virtual queue_op_status try_push(const value_type& x);
    virtual queue_op_status nonblocking_push(const value_type& x);

    virtual void push(value_type&& x);
    virtual queue_op_status wait_push(value_type&& x);
    virtual queue_op_status try_push(value_type&& x);
    virtual queue_op_status nonblocking_push(value_type&& x);

    virtual value_type value_pop();
    virtual queue_op_status wait_pop(value_type& x);
    virtual queue_op_status try_pop(value_type& x);
    virtual queue_op_status nonblocking_pop(value_type& x);
};
```

??6.2.3 Binary ends [`conqueues.tools.binends`]

Add a new section:

In addition to binary interfaces to queues, binary interfaces to ends are also useful.

```
template <typename Value>
alias queue_back = generic_queue_back< queue_base< Value > >;
template <typename Value>
alias queue_front = generic_queue_front< queue_base< Value > >;
```

??6.3 Managed Ends [`conqueues.tools.managed`]

Add a new section:

Automatically managing references to queues can be helpful when queues are used as a communication medium.

?.?.6.3.1 Class template `shared_queue_back` [`conqueues.tools.sharedback`]

Add a new section:

```
template <typename Value>
class shared_queue_back
{
public:
    typedef typename Value value_type;
    typedef value_type& reference;
    typedef const value_type& const_reference;

    typedef implementation-defined iterator;
    typedef const iterator const_iterator;

    shared_queue_back(const shared_queue_back& other);
    shared_queue_back& operator =(const shared_queue_back& other);

    void close();
    bool is_closed();
    bool is_empty();
    bool is_full();
    bool is_lock_free();

    iterator begin();
    iterator end();
    const_iterator cbegin();
    const_iterator cend();

    void push(const value_type& x);
    queue_op_status wait_push(const value_type& x);
    queue_op_status try_push(const value_type& x);
    queue_op_status nonblocking_push(const value_type& x);

    void push(value_type&& x);
    queue_op_status wait_push(value_type&& x);
    queue_op_status try_push(value_type&& x);
    queue_op_status nonblocking_push(value_type&& x);
};
```

The class template `shared_queue_back` implements `WaitingConcurrentQueueBack`

```
shared_queue_back(const shared_queue_back& other);
shared_queue_back& operator =(const shared_queue_back& other) = default;
```

Effects: Copy the pointer to the queue, but keep the back of the queue reference counted.

```
~shared_queue_back();
```

Effects: Destroys the queue back. If this is the last back reference, and there are no front references, destroy the queue. If this is the last back reference, and there are front references, close the queue.

?.?.6.3.2 Class template `shared_queue_front` [`conqueues.tools.front`]

Add a new section:

```
template <typename Queue>
class shared_queue_front
{
public:
    typedef typename Queue::value_type value_type;
    typedef value_type& reference;
    typedef const value_type& const_reference;

    typedef implementation-defined iterator;
    typedef const iterator const_iterator;

    shared_queue_front(Queue& queue);
    shared_queue_front(Queue* queue);
    shared_queue_front(const shared_queue_front& other) = default;
    shared_queue_front& operator =(const shared_queue_front& other) = default;
```

```

void close();
bool is closed();
bool is empty();
bool is full();
bool is lock free();
bool has queue();

iterator begin();
iterator end();
const iterator cbegin();
const iterator cend();

value_type value pop();
queue_op_status wait_pop(value_type& x);
queue_op_status try_pop(value_type& x);
queue_op_status nonblocking_pop(value_type& x);
};

```

The class template `shared_queue_front` implements `WaitingConcurrentQueueFront`

```

shared_queue_front(const shared_queue_front& other);
shared_queue_front& operator =(const shared_queue_front& other) = default;

```

Effects: Copy the pointer to the queue, but keep the front of the queue reference counted.

```

~shared_queue_front();

```

Effects: Destroys the queue front. If this is the last front reference, and there are no back references, destroy the queue. If this is the last front reference, and there are back references, close the queue.

6.3.3 Function template `share_queue_ends` [conqueues.tools.shareendsfront]

Add a new section:

```

template <typename Queue, typename ... Args>
std::pair< shared_queue_back<typename Queue::value_type>,
shared_queue_front<typename Queue::value_type> >
share_queue_ends(Args ... args);

```

Effects: Constructs a Queue with the given Args. Initializes a set of reference counters for that queue.

Returns: a pair consisting of one `shared_queue_back` and one `shared_queue_front`.

Revision History

This paper revises N3533 - 2013-03-12 as follows.

- Update links to source code.
- Add wording.
- Leave the name facility out of the wording.
- Leave the push-front facility out of the wording.
- Leave the reopen facility out of the wording.
- Leave the storage iterator facility out of the wording.

This N3532 revises N3434 = 12-0043 - 2012-01-14 as follows.

- Add more exposition.
- Provide separate non-blocking operations.
- Add a section on the lock-free queues.

- Argue against push-back operations.
- Add a cautionary note on the usefulness of `is_closed()`.
- Expand the cautionary note on the usefulness of `is_empty()`. Add `is_full()`.
- Add a subsection on element type requirements.
- Add a subsection on exception handling.
- Clarify ordering constraints on the interface.
- Add a subsection on a lock-free concrete queue.
- Add a section on content iterators, distinct from the existing streaming iterators section.
- Swap front and back names, as requested.
- General expository cleanup.
- Add an 'Revision History' section.

N3434 revised N3353 = 12-0043 - 2012-01-14 as follows.

- Change the inheritance-based interface to a pure conceptual interface.
- Put 'try' operations into a separate subsection.
- Add a subsection on non-blocking operations.
- Add a subsection on push-back operations.
- Add a subsection on queue ordering.
- Merge the 'Binary Interface' and 'Managed Indirection' sections into a new 'Conceptual Tools' section. Expand on the topics and their rationale.
- Add a subsection to 'Conceptual Tools' that provides for type erasure.
- Remove the 'Synopsis' section.
- Add an 'Implementation' section.